Mälardalen University

MAA069
Numerical Methods with MATLAB

Spring 2026

Laboratory report

Author(s):
Andrii Pozhylenkov
Dmytro Kuzko

Lab Group: 12

Instructor:
Maksat Ashyralyyev

April 2, 2026

Contents

1 Solving Nonlinear Algebraic Equation

The trajectory of a ball thrown by a ball launcher is defined by the (x, y) coordinates and it is modelled by the following equation:

$$y = x \tan\theta - \frac{gx^2}{2v^2 \cos^2\theta} + y_0 \tag{1}$$

where θ denotes the initial angle, v denotes the initial velocity, and $g = 9.81$ m/s^2 is the gravitational constant.

Consider the problem of finding the initial angle θ so that the ball is supposed to hit the target located on the ground at the distance of 90 m from the launcher given that the initial velocity is $v = 30$ m/s and the launcher stands 1 m above the ground. It results in a nonlinear algebraic equation with two distinct roots in the interval $[0, \pi/2]$.

1.1 Solution & Results

1.1.1 Extract non-linear equation

Step 1: identify boundary conditions and system parameters

Based on the problem description, the following parameters and conditions are given for the moment the ball hits the target:

- Final horizontal position (x): 90 m
- Final vertical position (y): 0 m (target is located on the ground)
- Initial velocity (v): 30 m/s
- Initial height (y_0): 1 m
- Gravitational acceleration (g): 9.81 m/s^2

Step 2: substitute parameters into the model

Inserting the given values into the trajectory equation gives us:

$$0 = 90 \tan\theta - \frac{9.81 \cdot 90^2}{2 \cdot 30^2 \cos^2\theta} + 1 \tag{2}$$

Step 3: simplify the mathematical expression

Evaluate the constant term within the equation:

$$\frac{9.81 \cdot 8100}{2 \cdot 900} = \frac{79461}{1800} = 44.145 \tag{3}$$

Substitute the simplified constant back into the equation:

$$0 = 90 \tan\theta - \frac{44.145}{\cos^2\theta} + 1 \tag{4}$$

Step 4: formulate the final equation

To get a non-linear algebraic equation, we need to simplify current equation. For that we can apply the trigonometric equation:

$$\frac{1}{\cos^2\theta} = 1 + \tan^2\theta \tag{5}$$

With that let's use in the current equation in simplified form:

$$0 = 90\tan\theta - 44.145(1 + \tan^2\theta) + 1 \tag{6}$$

$$0 = -44.145\tan^2\theta + 90\tan\theta - 43.145 \tag{7}$$

Final non-linear algebraic equation:

$$f(\theta) = -44.145\tan^2\theta + 90\tan\theta - 43.145 = 0 \tag{8}$$

1.1.2 Proof of single existing roots on the given intervals

Step 1: Check the function signs at the interval edges
We need to calculate the value of our function

$$f(\theta) = -44.145\tan^2\theta + 90\tan\theta - 43.145 \tag{9}$$

at the boundaries of our intervals.

First interval $[0, \pi/4]$

At $\theta = 0$:
The value of $\tan(0) = 0$. The result:

$$f(0) = -43.145 \tag{10}$$

The result is **negative**.

At $\theta = \pi/4$:
The value of $\tan(\pi/4) = 1$. The result:

$$f(\pi/4) = -44.145(1)^2 + 90(1) - 43.145 = 2.71 \tag{11}$$

The result is **positive**.

Result: The function has different signs on the edges of the interval $[0, \pi/4]$.

Second interval $[\pi/4, \pi/2)$

At $\theta = \pi/4$, we already know the value is positive and equals to 2.71. As θ gets closer to $\pi/2$ - $\tan(\theta)$ goes to infinity. The squared term $-44.145 \times \tan^2\theta$ grows very fast and is also negative. This gives us that the function value goes to $-\infty$.

Result: The function has different signs on the edges of the interval $[\pi/4, \pi/2]$.

Step 2: Explain the function continuity

Our function $f(\theta)$ is a polynomial of $\tan\theta$. The tangent function is continuous for all points starting from 0 up to (but not including) $\pi/2$. That gives us that our function $f(\theta)$ continuous on both $[0, \pi/4]$ and $[\pi/4, \pi/2)$ intervals.

Step 3: Show monotonicity using the derivative

The derivative of our function is:

$$f'(\theta) = -88.29 \tan\theta \frac{1}{\cos^2\theta} + 90 \frac{1}{\cos^2\theta} \tag{12}$$

$$f'(\theta) = \frac{1}{\cos^2\theta}(90 - 88.29 \tan\theta) \tag{13}$$

We use the first derivative to show the function only decreasing or increasing on the interval when crossing zero. This monotonicity proves there is only one root per interval.

First interval $[0, \pi/4]$

Here, $\tan\theta$ is between 0 and 1. This makes the term $(90 - 88.29 \tan\theta)$ always positive. Because $f'(\theta) > 0$ on the whole interval, the function is strictly increasing on that interval.

Second interval $[\pi/4, \pi/2)$

Consider $f'(\theta) = 0$ to find the highest point. This peak happens when $\tan\theta \approx 1.019$. This angle is just a little past $\pi/4$. Between $\pi/4$ and $\pi/2$, the sign of the derivative actually changes. Right at $\pi/4$, the derivative is still positive. But at $\theta = \pi/4$, we already know the value is positive and equals to 2.71. It means that function starting from the positive value, then slightly increasing. It hits the peak. After the peak, the term $(90 - 88.29 \tan\theta)$ drops below zero. This makes the whole derivative negative. Because the derivative stays negative after the peak, the graph keeps going down until it crosses the x-axis.

Step 4: Final conclusion

By the **Intermediate Value Theorem**, a continuous function that changes sign must cross zero at least once.

In the first interval $[0, \pi/4]$, the function is continuous, changes sign, and is strictly increasing. This proves there is exactly one root here.

In the second interval $[\pi/4, \pi/2)$, the function is continuous, changes sign, and strictly decreases. This proves there is exactly one root here as well.

1.1.3 Theory of the Bisection Method

The Bisection Method [1, page 25] is a simple and reliable way to find the root of an equation. It works by dividing an interval in half over and over again. You do not need a full plot of the function to use it.

Step-by-Step Process:

- **Start with an interval:** Choose a starting interval $[a, b]$. The function values at the edges, $f(a)$ and $f(b)$, must have opposite signs. This means $f(a)f(b) < 0$. This guarantees that a root exists inside the interval.
- **Find the midpoint:** Calculate the center point, c, using the formula:

$$c = \frac{a + b}{2}$$

- **Check the midpoint:** Calculate the function value at the new point, $f(c)$. If $f(c) = 0$, you have found the exact root and the work is done.
- **Update the interval:** If $f(c)$ is not zero, look at the signs to make a new, smaller interval:
  - If $f(a)$ and $f(c)$ have opposite signs ($f(a)f(c) < 0$), the root is in the left half. The new interval is $[a, c]$.
  - If $f(b)$ and $f(c)$ have opposite signs ($f(b)f(c) < 0$), the root is in the right half. The new interval is $[c, b]$.
- **Repeat:** The new interval is exactly half the size of the old one. Repeat these steps to make the interval smaller and locate the root more accurately.

Think of it like this: you start with a big search area, the interval $[a, b]$. Every time you take a step in the Bisection Method, you are literally chopping that area in half.

After doing that n times, your search area has shriveled down to a tiny width of $\frac{b-a}{2^n}$. When you pick the midpoint $x_c = \frac{a_n+b_n}{2}$ as your final guess, you know the real answer r is trapped somewhere in that tiny space.

If you have a close enough target - which we call the tolerance ϵ - you just need to keep bisecting until that tiny interval is smaller than or equal to your limit:

$$\text{Solution error} = \frac{b - a}{2^n} \leq \epsilon \tag{14}$$

where ϵ is a given tolerance

1.1.4 Application of the Bisection Method

In this section we will apply Bisection Method in order to solve non-linear equation from our initial problem.

Assess number of iterations

Based on the equation (14) we can infer the number of iterations needed for our problem. Here is inequality for given task with the precission of 10^{-15}:

$$n > \log_2\left(\frac{\pi}{4} \cdot 10^{15}\right)$$
$$n > 49.4809$$

Minimum number of iterations: 50

Order of convergence

For the Bisection Method, the interval length is reduced by a factor of 2 at every step.

$$e_{n+1} \approx \frac{1}{2} e_n$$

This means that the Bisection Method has linear convergence. The error decreases at a constant rate and each iteration improves the accuracy by approximately one additional binary digit.

Results obtained from the code

- Approximated initial angle: $\theta \approx 0.6567089205736349$
- Number of iterations: $n = 50$

1.1.5 Theory of the Newton-Raphson Method

Newton's Method, also called the Newton–Raphson Method [1, page 53], usually converges much faster than the linearly convergent methods we have seen previously. The geometric picture of Newton's Method is shown in Figure 1.8. To find a root of $f(x) = 0$, a starting guess x_0 is given, and the tangent line to the function f at x_0 is drawn. The tangent line will approximately follow the function down to the x-axis toward the root. The intersection point of the line with the x-axis is an approximate root, but probably not exact if f curves. Therefore, this step is iterated.

From the geometric picture, we can develop an algebraic formula for Newton's Method. The tangent line at x_0 has slope given by the derivative $f(x_0)$. One point on the tangent line is $(x_0, f(x_0))$. The point-slope formula for the equation of a line is:

$$y - f(x_0) = f'(x_0)(x - x_0) \tag{15}$$

so that looking for the intersection point of the tangent line with the x-axis is the same as substituting $y = 0$ in the line:

$$f'(x_0)(x - x_0) = 0 - f(x_0) \tag{16}$$

$$x - x_0 = -\frac{f(x_0)}{f'(x_0)} \tag{17}$$

$$x = x_0 - \frac{f(x_0)}{f'(x_0)} \tag{18}$$

Solving for x gives an approximation for the root, which we call x_1. Next, the entire process is repeated, beginning with x_1, to produce x_2, and so on, yielding the following iterative formula:

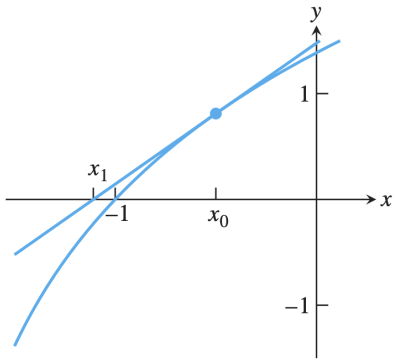

Figure 1: One step of Newton's Method. Starting with x_0, the tangent line to the curve $y = f(x)$ is drawn. The intersection point with the x-axis is x_1, the next approximation to the root.

$$x_0 = \text{initial guess} \tag{19}$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad \text{for} \quad i = 0, 1, 2, \ldots \tag{20}$$

1.1.6 Application of the Newton–Raphson

In this section we will apply Newton–Raphson in order to solve non-linear equation from our initial problem.

As an initial guesses we will use $p_0 = 0$ and $p_1 = \pi/4$

Results obtained from the code

- Approximated initial angle:
  - $p_0 = 0$ approximation: $\theta \approx 0.6567089205736348$
  - $p_1 = \pi/4$ approximation: $\theta \approx 0.6567089205736348$
- Number of iterations:
  - $p_0 = 0$ iterations: $n = 7$
  - $p_1 = \pi/4$ iterations: $n = 8$
- Error comparing to the result obtained in Bisection Method part:
  - $p_0 = 0$ error: 1.110223024625157e-16
  - $p_1 = \pi/4$ error: 1.110223024625157e-16

Calculation of the order of convergence

To measure how fast a numerical method finds the exact root, we calculate the order of convergence, α. Here is the step-by-step derivation of the formula for α.

Step 1: define the error ratio between each other [1, page 35-37, 52-54]

The order of convergence looks from the error at the next step (e_{n+1}) towards the error at the current step (e_n). This ratio is defined with the additional constant term C. The starting formula is:

$$|e_{n+1}| \approx C|e_n|^{\alpha}$$

Step 2: use logarithmic properties for separating

We want to find α, but the problem at the moment that we do not know C. To avoid this, we can take the logarithm (ln) of both sides of the equation. This gives us next equation:

$$\ln(|e_{n+1}|) \approx \ln(C) + \alpha \ln(|e_n|)$$

Step 3: equation for the previous step

$$\ln(|e_n|) \approx \ln(C) + \alpha \ln(|e_{n-1}|)$$

Step 4: subtract to cancel out the constant

We subtract the second equation from the first equation. Because both equations have this term $\ln(C)$, this constant is canceled out from the equation:

$$\ln(|e_{n+1}|) - \ln(|e_n|) \approx \alpha\big(\ln(|e_n|) - \ln(|e_{n-1}|)\big)$$

Step 5: find the final formula

Using standard logarithm rules to group the terms - we isolate α by dividing both sides. Final formula will look like this

$$\alpha \approx \frac{\ln(|e_{n+1}|/|e_n|)}{\ln(|e_n|/|e_{n-1}|)} \tag{21}$$

With that we can look at α values on the latest steps observing the order of convergence.

For Newton-Raphson method, for both p_0 and p_1 initial guesses the order of convergence is quadratic. We can also observe that based on the estimation on every step:

For p_0:

- step 3: $\alpha \approx 1.4262$
- step 4: $\alpha \approx 1.6712$
- step 5: $\alpha \approx 1.9645$

- step 6: $\alpha \approx 1.999$
- step 7: $\alpha \approx 1.1319$

For p_1:

- step 3: $\alpha \approx 2.3559$
- step 4: $\alpha \approx 1.4325$
- step 5: $\alpha \approx 1.6651$
- step 6: $\alpha \approx 1.9633$
- step 7: $\alpha \approx 1.9989$
- step 8: $\alpha \approx 1.1573$

On the last step the value of Alpha significantly decreasing because of the limited machine precision. The errors are sufficiently small that machine evaluating their ratio as almost the same.

1.1.7 Theory of the Secant Method

The Secant Method [1, page 61] is a popular technique to find the root of an equation. It is very similar to Newton's Method, but it is easier to use because it does not require calculating the exact derivative of the function.

How it Works:

Newton's Method uses a tangent line at one point to find the next guess. The Secant Method replaces this with a "secant line." A secant line is a straight line drawn through the two most recent guess points on the function's graph. The point where this line crosses the zero level becomes the new guess.

The Formula:

To calculate the next approximation (x_{i+1}), the method uses the difference between the last two points instead of a derivative:

$$x_{i+1} = x_i - \frac{f(x_i)(x_i - x_{i-1})}{f(x_i) - f(x_{i-1})} \tag{22}$$

1.1.8 Application of the Secant

In this section we will apply Secant in order to solve non-linear equation from our initial problem.

As an initial guesses we will use $p_0 = 0$ and $p_1 = \pi/4$

Results obtained from the code

- Approximated initial angle: $\theta \approx 0.6567089205736352$
- Number of iterations: $n = 10$

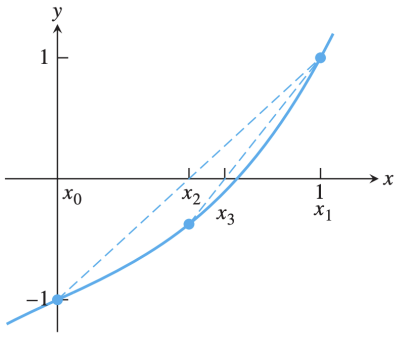

Figure 2: Two steps of the Secant Method. Starting with $x_0 = 0$ and $x_1 = 1$, the Secant Method iterates are plotted along with the secant lines.

Calculation of the order of convergence

To calculate the order of convergence we used the same logic and formulas that were described in the previous section.

For Secant method, the order of convergence was "super" linear ($\alpha \approx 1.6125$). We can also observe that based on the estimation on every step:

- step 3: $\alpha \approx$ -0.23153
- step 4: $\alpha \approx 6.131$
- step 5: $\alpha \approx 0.58335$
- step 6: $\alpha \approx 2.3024$
- step 7: $\alpha \approx 1.5636$
- step 8: $\alpha \approx 1.6302$
- step 9: $\alpha \approx 1.6125$
- step 10: $\alpha \approx$ Inf

On the last step the value of α significantly decreasing because of the limited machine precision.

1.2 Conclusions

- **Results:** we successfully found the correct result. Obtained the same results with all of the methods. All of the methods solved the given non-linear algebraic equation
- **Bisection method:** this method demonstrated itself as a slow but safe method which will always give you a result if the root is in the interval. Has a linear order of convergence

- **Newton-Raphson method:** this method demonstrated itself as the fastest one with quadratic convergence. However, it requires a good initial guesses.
- **Secant method:** this method is something in the middle between Newton-Raphson and Bisection. However, it has significantly smaller number of iterations required than Bisection. Also it does not require derivative, which is a good trade-off.
- **Machine precision issues:** while calculating the order of convergence we faced the machine precision problem on the last steps. That caused false values for the estimated alpha on last steps.
- **Errors:** difference between methods is significantly small comparing to each other. Overall, all methods showed great results with given initial parameters and accuracy.

2 Numerical Differentiation & Numerical Integration

Consider the problem of computing the integral

$$I = \int_1^2 \ln(f''(x))\,dx, \tag{23}$$

where smooth function f is not known but its values over the interval $[1, 2]$ are given in the following table:

x_k	1	1.1	1.2	1.3	1.4	1.5	1.6	1.7	1.8	1.9	2
$f(x_k)$	0.4	0.752	1.236	1.864	2.648	3.6	4.732	6.056	7.584	9.328	11.3

2.1 Solution & Results

2.1.1 Calculation of the Second Derivative

To calculate the second derivative of discrete data, we use finite difference formulas. These formulas estimate the derivative at a specific point by using the function values at neighboring points. The constant distance between these points is called h. We must use three different formulas to process the entire dataset.

1. **Central Difference Formula** [2, page 329]
   For all points in the middle of our dataset, we use the central difference formula. This is the most standard method because it uses points on both the left and right sides of the target point.

   $$f''(x_i) \approx \frac{f_{i-1} - 2f_i + f_{i+1}}{h^2}$$

2. **Forward Difference Formula** [2, page 333]
   For the very first point in our dataset, we cannot look backward because there are no previous points. Instead, we use a one-sided forward difference formula. This looks at the current point and the next three points ahead.

   $$f''(x_0) \approx \frac{2f_0 - 5f_1 + 4f_2 - f_3}{h^2}$$

3. **Backward Difference Formula** [2, page 333]
   Similarly, for the very last point in the dataset, we cannot look forward. We use a one-sided backward difference formula. This looks at the current point and the three previous points behind it.

   $$f''(x_0) \approx \frac{2f_0 - 5f_{-1} + 4f_{-2} - f_{-3}}{h^2}$$

Calculated second derivative vector:
The calculations produced the following 11 values (rounded for clarity):

f''	12	13.2	14.4	15.6	16.8	18	19.2	20.4	21.6	22.8	24

Natural logarithm of the second derivative vector ($\ln(f'')$):
Next, we applied a natural logarithm to each element in the f'' vector, which produced the following values:

$\ln(f'')$	2.4849	2.5802	2.6672	2.7473	2.8214	2.8904	2.9549	3.0155	3.0727	3.1268	3.1781

2.1.2 Numerical Integration

To calculate the definite integral from our discrete data points, we use numerical integration methods.

1. **Trapezoidal Rule** [1, page 255]
   This method approximates the area under the function by connecting the data points with straight lines. This creates a series of trapezoids.

   - **Formula:** To find the total area, we sum the areas of all the small trapezoids. The formula multiplies the endpoints by 1 and all the middle points by 2:
   $$T(f,h) = \frac{h}{2}(f_0 + 2f_1 + 2f_2 + \cdots + 2f_{M-1} + f_M) \tag{24}$$
   - **Order of convergence:** The Trapezoidal Rule has a theoretical order of convergence of 2, meaning the global error is proportional to h^2.
   - **Result:** Applying this rule to our previously calculated data vector ($\ln(f'')$), the code produced the following integral value:
   $$\text{Trapezoidal Sum} = 2.870784586533755$$

2. **Simpson's Rule** [1, page 257]
   Instead of straight lines, this method uses parabolas (quadratic curves) to connect sets of three adjacent points. This provides a much smoother fit to the data.

   - **Formula:** This method requires an even number of subintervals ($2M$). The formula uses an alternating pattern of 4 and 2 for the middle points:
   $$S(f,h) = \frac{h}{3}(f_0 + 4f_1 + 2f_2 + 4f_3 + \cdots + 2f_{2M-2} + 4f_{2M-1} + f_{2M}) \tag{25}$$

- **Order of convergence:** Simpson's Rule has a theoretical order of convergence of 4, meaning the global error is proportional to h^4.
- **Result:** Applying Simpson's Rule to our data vector ($\ln(f'')$), the code produced the following integral value:

$$\text{Simpson Sum} = 2.871200053592807$$

2.1.3 Error Analysis and Comparison

To check the accuracy of our numerical methods, we compared our calculated sums to the exact given mathematical result.

1. Exact Value and Calculated Errors
The exact value of the integral is 2.871201010907891.
By calculating the difference between this exact value and our numerical sums, we found the absolute error for each method:

- **Trapezoidal Error (`trap_err`):** $4.164243741358042 \times 10^{-4}$
- **Simpson's Error (`simp_err`):** $9.573150840935796 \times 10^{-7}$

2. Comparison of Errors
When we compare the two errors, it is clear that Simpson's Rule is much more accurate. The error for the Trapezoidal Rule is around 4.16×10^{-4}, while the error for Simpson's Rule is much smaller, around 9.57×10^{-7}. This means that the error from Simpson's Rule is hundreds of times smaller than the error from the Trapezoidal Rule.

3. Explanation
We can explain this large difference in accuracy by looking at how each method works mathematically.

- **Trapezoidal Rule** connects points with straight lines. It has an order of convergence of 2. Usually used when points are not evenly distributed or the number of subintervals is odd (number of points is even).
- **Simpson's Rule** connects points using curved parabolas. It has a higher order of convergence of 4. Requires an odd number of points. Usually used for evenly distributed points.

Since the best conditions for Simpson's are met (odd number of evenly distributed points) and it has a higher order, its error shrinks much faster, giving much better results.

2.2 Conclusions

Based on the calculations and error analysis performed in this section, we can draw the following conclusions:

- **Comparison of integration methods:** Both the Trapezoidal Rule and Simpson's Rule provided good approximations of the true integral area. However, their levels of accuracy were very different.
- **Advantage of Simpson's Rule:** The error analysis proved that Simpson's Rule (9.57×10^{-7}) was significantly more accurate than the Trapezoidal Rule (4.16×10^{-4}).
- **Reasons for different accuracy:** This difference in accuracy is expected based on the mathematical theory. Because our dataset had an odd number of evenly spaced points (11 points, 10 subintervals), it was the perfect setup for Simpson's Rule. Since Simpson's Rule uses curved parabolas ($O(h^4)$ error) rather than the straight lines of the Trapezoidal Rule ($O(h^2)$ error), it fits the data much better and produces a much smaller error.

3 Interpolation & Curve Fitting

The concentration of *E. coli* bacteria in a swimming area was measured after a storm and the data is given in the following table.

Time t (hours)	Concentration c (CFU/100 mL)
4	1600
8	1320
12	1000
16	890
20	650
24	560

Consider the problem of estimating the concentration of *E. coli* bacteria at the end of the storm, i.e. at time $t = 0$.

3.1 Solution & Results

3.1.1 Theory of Newton Polynomial Interpolation

Newton Interpolation Method is a way to construct a mathematical curve that passes exactly through a specific set of data points. If we have $N + 1$ distinct data points, this method creates a unique polynomial of degree N or less.

To build this polynomial, we use a mathematical tool called divided differences.

1. Divided Differences [2, page 223]
Divided differences are calculated step-by-step from our given data points (x_k, y_k). Zeroth-order difference is simply the function value at that point:

$$f[x_k] = f(x_k)$$

The first-order difference uses two neighboring points:

$$f[x_{k-1}, x_k] = \frac{f[x_k] - f[x_{k-1}]}{x_k - x_{k-1}}$$

For higher orders, we use a recursive rule. We subtract two lower-order differences and divide by the total distance between the outermost x values:

$$f[x_{k-j}, \ldots, x_k] = \frac{f[x_{k-j+1}, \ldots, x_k] - f[x_{k-j}, \ldots, x_{k-1}]}{x_k - x_{k-j}}$$

2. Divided-Difference Table [2, page 224]
In practice, these calculations are organized into a table. The first two columns contain the given x and $f(x)$ values. Each new column to the right calculates the next level of divided differences using the values from the column before it. The most important numbers in this table are the ones located on the main diagonal.

3. The Newton Polynomial Formula [2, page 220 - 221]
Once the table is complete, we can write the final polynomial. The Newton form of the polynomial is written as:

$$P_N(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_N(x - x_0)(x - x_1)\ldots(x - x_{N-1}) \tag{26}$$

The coefficients $(a_0, a_1, \ldots, a_N)$ are the diagonal entries from our divided-difference table.

3.1.2 Application of Newton Interpolation Method

1. Divided Differences Matrix A
Based on input t and c values we can construct the Divided Differences table (lower triangular matrix):

$$A = \begin{bmatrix} 1600 & 0 & 0 & 0 & 0 & 0 \\ 1320 & -70 & 0 & 0 & 0 & 0 \\ 1000 & -80 & -1.25 & 0 & 0 & 0 \\ 890 & -27.5 & 6.5625 & 0.6510 & 0 & 0 \\ 650 & -60 & -4.0625 & -0.8854 & -0.0960 & 0 \\ 560 & -22.5 & 4.6875 & 0.7292 & 0.1009 & 0.0098 \end{bmatrix}$$

Coefficient Vector p
The coefficients of Newton polynomial correspond to the diagonal elements of matrix A. The decimal values:

$$p = \begin{bmatrix} 1600 \\ -70 \\ -1.25 \\ 0.6510416667 \\ -0.0960286458 \\ 0.0098470052 \end{bmatrix}$$

Interpolation Nodes

$$t = \begin{bmatrix} 4 & 8 & 12 & 16 & 20 & 24 \end{bmatrix}$$

2. Construction of the Newton Polynomial
The formula of the Newton interpolation polynomial for the given nodes has the following form:

$$P(t) = p_0 + p_1(t - t_0) + p_2(t - t_0)(t - t_1) + \cdots + p_n(t - t_0)\ldots(t - t_{n-1})$$

Tuning in values of the nodes t_i and the decimal coefficients p_i, we obtain

following polynomial:

$$\begin{aligned} P(x) = 1600 &- 70(x-4) - 1.25(x-4)(x-8) \\ &+ 0.6510416667(x-4)(x-8)(x-12) \\ &- 0.0960286458(x-4)(x-8)(x-12)(x-16) \\ &+ 0.0098470052(x-4)(x-8)(x-12)(x-16)(x-20) \end{aligned} \tag{27}$$

3. Normalized Form of the Polynomial
To reduce our polynomial to the classic form $P(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_0$, the brackets are expanded and terms are grouped together.
Result in decimal form:

$$\begin{aligned} P(x) = 0.0098470052x^5 &- 0.6868489583x^4 + 17.8841145833x^3 \\ &- 212.4479166667x^2 + 1057.5833333333x - 210 \end{aligned} \tag{28}$$

3.1.3 Graphical Analysis and the Limits of Extrapolation

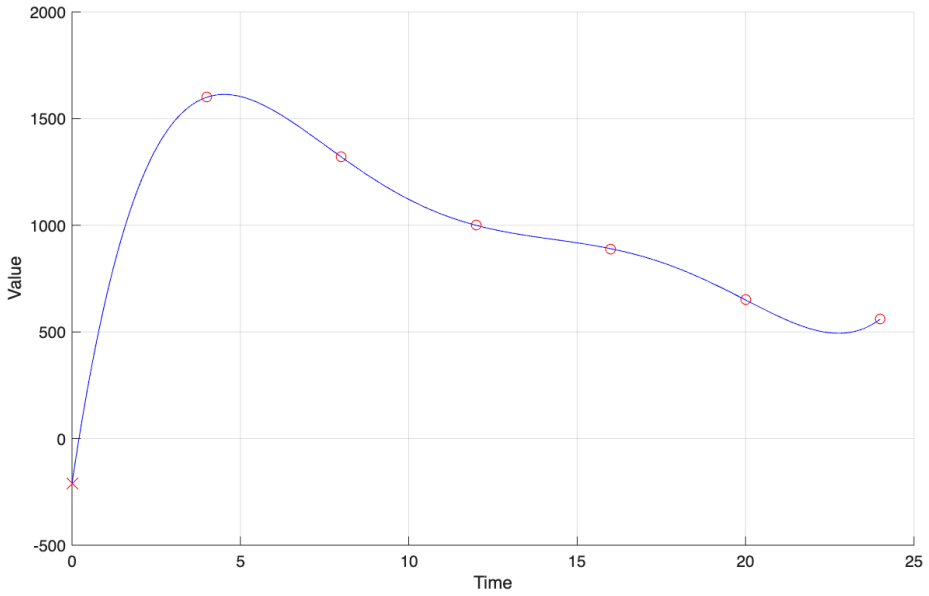

Figure 3: Graph of obtained polynomial in the interval [0, 24] and the data points marked as circles where $t = 0$ (extrapolation point) marked as X-mark

1. Interpolation Results
The generated graph illustrates a 5th-degree polynomial curve (in blue) fitted to the discrete measurements of bacteria concentration over time. The curve captures a physically realistic trend during this timeframe: an initial increase

in infection following the storm. This is followed by a gradual weakening as the system begins to clear.

2. Polynomial Extrapolation
Extrapolation involves using our mathematical model to predict values outside of the given data (specifically, before hour 4 and after hour 24). When evaluating the polynomial at $x = 0$, the model returns a concentration of -210.

This negative result occurs for primary reason:

Polynomials Limitations: High-degree polynomials (like our 5th-degree model) are unstable outside their interpolation interval. They tend to oscillate once they pass the initial data points. The algorithm's objective was to minimize the error between the known data points, not to understand the broader context of the function.

3.1.4 Theory of Least-Squares Fitting

Curve fitting is a mathematical process used to construct a continuous function that best represents a set of discrete data points. Unlike interpolation (which forces the curve to pass exactly through every point), the least-squares method finds a "best fit" model by minimizing the overall error (in this case RMS) between the calculated curve and the actual data.

1. Linear Curve Fitting (The Least-Squares Line) [2, page 255]
If we have a dataset of N distinct points (x_k, y_k), the simplest approach is to fit a straight line. The equation for this line is:

$$y = Ax + B$$

To find the optimal coefficients A (the slope) and B (the y-intercept) that minimize the error, we must solve a specific linear system known as the normal equations. By summing the combinations of our data points, we set up the following two equations:

$$\left(\sum_{k=1}^{N} x_k^2\right) A + \left(\sum_{k=1}^{N} x_k\right) B = \sum_{k=1}^{N} x_k y_k$$
$$\left(\sum_{k=1}^{N} x_k\right) A + NB = \sum_{k=1}^{N} y_k$$

Solving this system simultaneously provides the exact values for A and B to construct the best-fit line.

2. Exponential Curve Fitting (Data Linearization) [2, page 263]
Often, physical processes do not follow a straight line and are better modeled by an exponential curve:

$$y = Ce^{Ax}$$

Because this equation is non-linear, we cannot apply the normal equations directly. Instead, we must use a technique called data linearization.

Step 1: Logarithmic Transformation
We take the natural logarithm of both sides of the equation to break down the exponential term:

$$\ln(y) = Ax + \ln(C)$$

Step 2: Change of Variables
We introduce new variables to rewrite the equation in a standard linear format. Let:

$$Y = \ln(y)$$
$$X = x$$
$$B = \ln(C)$$

This transforms our complex equation into a simple linear relation:

$$Y = AX + B$$

Step 3: Applying the Normal Equations
We map our original data points (x_k, y_k) to the new linearized plane, creating transformed points $(X_k, Y_k) = (x_k, \ln(y_k))$. We then feed these X and Y values into the standard normal equations (from Part 1) to solve for A and B.

Step 4: Reversing the Transformation
Once the linear system is solved, the parameter A is ready to use. However, B is still in its logarithmic form. To find the original coefficient C for our exponential model, we compute the inverse logarithm:

$$C = e^B$$

This process allows us to accurately fit complex, non-linear curves using standard, reliable linear algebra tools.

3. RMS [2, page 253]
The Root-Mean-Square error is a very common and useful way to find the overall error. To calculate it, you square every residual, find the average of all those squared values, and finally take the square root of that average.

The formula for the RMS error (E) for N data points is:

$$E(f) = \left(\frac{1}{N} \sum_{k=1}^{N} |f(x_k) - y_k|^2 \right)^{1/2} \tag{29}$$

In curve fitting, the RMS error is often preferred because squaring the errors strongly penalizes points that are very far away from the curve.

3.1.5 Results and Comparison of Curve Fitting Models

1. Calculation of Coefficients
To create our models, we applied the least-squares theory using our MATLAB code.

For the linear model, passing the time (t) and concentration (c) arrays into our solver function yielded the following coefficients for the equation $c(t) = At + B$:

$$A = -52.2857$$
$$B = 1735.3333$$

Thus, the resulting linear model is:

$$c(t) = -52.2857t + 1735.3333$$

For the exponential model $c(t) = Ce^{At}$, we first linearized the data by taking the natural logarithm of the concentration array. Solving the normal equations for the transformed data $(t, \ln(c))$ gave the intermediate linear coefficients for the relation $\ln(c) = At + B$:

$$A = -0.0535$$
$$B = 7.5936$$

To find the final exponential constant C, we transformed the intermediate intercept B back using the exponential function, exactly as described in the theory:

$$C = e^B = e^{7.5936} = 1985.4366$$

With both parameters calculated, our final exponential model is:

$$c(t) = 1985.4366e^{-0.0535t}$$

2. Graphical and Numerical Results

We used both models to extrapolate backward to $t = 0$. This gives us an estimate of the bacteria concentration at the exact moment the storm happened. We also calculated the Root Mean Square (RMS) error. This number tells us how far off our calculated line is from the real data points (a lower number is better).

The program gave these results:

- **Linear Model:** The estimated concentration at $t = 0$ is 1735.33. The RMS error is 64.64.
- **Exponential Model:** The estimated concentration at $t = 0$ is 1985.44. The RMS error is 31.39.

3. Comparison

When we compare two methods, exponential model is clearly superior to linear.

- **Error Analysis:** The RMS error for the exponential curve (31.39) is more than two times smaller than the error for the straight line (64.64). This proves the green curve fits our dataset much better mathematically.

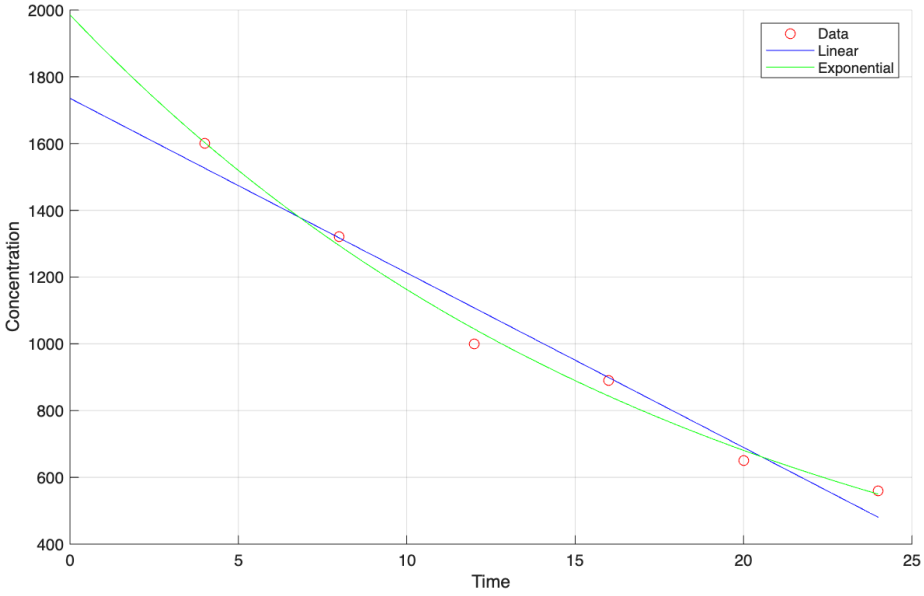

Figure 4: Resulting least squares line, the least squares exponential curve and the given data points

- **Visual:** Looking at the graph, the blue straight line completely misses the middle points and the final point. The green curve naturally bends and passes very close to almost all the red circles.
- **Reality:** In biology, natural decay processes usually follow an exponential curve. When there is a lot of bacteria, the concentration drops quickly. As the water gets cleaner, the rate of decay slows down. The linear model is flawed because it assumes the bacteria disappear at the exact same constant speed forever.

3.2 Conclusions

Based on the analysis of the different mathematical models in this section, we can draw the following conclusions regarding the initial bacteria concentration:

- **Failure of Polynomial extrapolation:** While the 5th-degree Newton polynomial perfectly connected our known data points, it completely failed at predicting the past. It produced an impossible negative concentration (-210) for time zero. This proves that high-degree polynomials are highly unstable and should not be used for extrapolation outside the given data range.
- **Inaccuracy of the Linear model:** The linear least-squares model assumed the bacteria died at a constant speed. This resulted in a poor fit

with a high error (RMS = 64.64) because a straight line does not reflect how real biological decay works.

- **Superiority of the Exponential model:** The exponential model performed much better. It had an error that was more than two times smaller (RMS = 31.39) and correctly matched the natural biological process, where the rate of bacterial decay naturally slows down as the water gets cleaner.
- **Final Estimation:** Because the exponential curve is the most mathematically accurate and physically realistic model for this dataset, we conclude that our best estimate for the initial *E. coli* concentration right after the storm is 1985.44 CFU/100 mL.

4 Numerical Solution of Differential Equations

Consider a spread of the infectious disease in the isolated population (no migration). The disease is transmitted from human to human and gives lifelong immunity after recovery. There is no vaccine available. Under these assumptions, a simple mathematical model, known in epidemiology as SIRD model[1], is defined by the following system of differential equations

$$\begin{cases} S'(t) = -\dfrac{\beta}{N} I(t) S(t) \\ I'(t) = \dfrac{\beta}{N} I(t) S(t) - \alpha I(t) - \gamma I(t) \\ R'(t) = \alpha I(t) \\ D'(t) = \gamma I(t) \end{cases} \tag{30}$$

where $S(t)$ denotes the number of susceptible individuals, $I(t)$ denotes the number of infected individuals, $R(t)$ denotes the number of recovered individuals, $D(t)$ denotes the number of deceased individuals, β is the infection rate, α is the recovery rate, γ is the mortality rate, and $N = S + I + R + D$. Assume that $S(0) = 997$, $I(0) = 3$, $R(0) = 0$, $D(0) = 0$, and $t \in [0, 180]$.

4.1 Solution & Results

To apply numerical methods such as Heun's method, we first need to express the given system of differential equations as an Initial Value Problem (IVP) in a general vector form:

$$\begin{cases} \frac{dY}{dt} = F(t, Y(t)), \quad 0 \le t \le T, \\ Y(0) = Y_0 \end{cases} \tag{31}$$

Based on our SIRD model, the state vector $Y(t)$, the initial condition vector Y_0, and the vector function $F(t, Y(t))$ are extracted and defined as follows:

$$Y(t) = \begin{bmatrix} S(t) \\ I(t) \\ R(t) \\ D(t) \end{bmatrix}, \quad Y_0 = \begin{bmatrix} S(0) \\ I(0) \\ R(0) \\ D(0) \end{bmatrix} = \begin{bmatrix} 997 \\ 3 \\ 0 \\ 0 \end{bmatrix} \tag{32}$$

$$F(t, Y(t)) = \begin{bmatrix} -\frac{\beta}{N} I(t) S(t) \\ \frac{\beta}{N} I(t) S(t) - (\alpha + \gamma) I(t) \\ \alpha I(t) \\ \gamma I(t) \end{bmatrix} \tag{33}$$

This general vector formulation allows us to substitute $F(t, Y(t))$ directly into the iterative formulas of the numerical solvers.

4.1.1 Task (a): MATLAB *ode45*

Initial vector:

- $S(0) = 997$;
- $I(0) = 3$;
- $R(0) = 0$;
- $D(0) = 0$;
- Interval $= [0, 180]$ days.

Parameters:

- Total population $= 1000$;
- Infection rate $\beta = 0.4$;
- Recovery rate $\alpha = 0.035$;
- Mortality rate $\gamma = 0.005$.

Obtained results

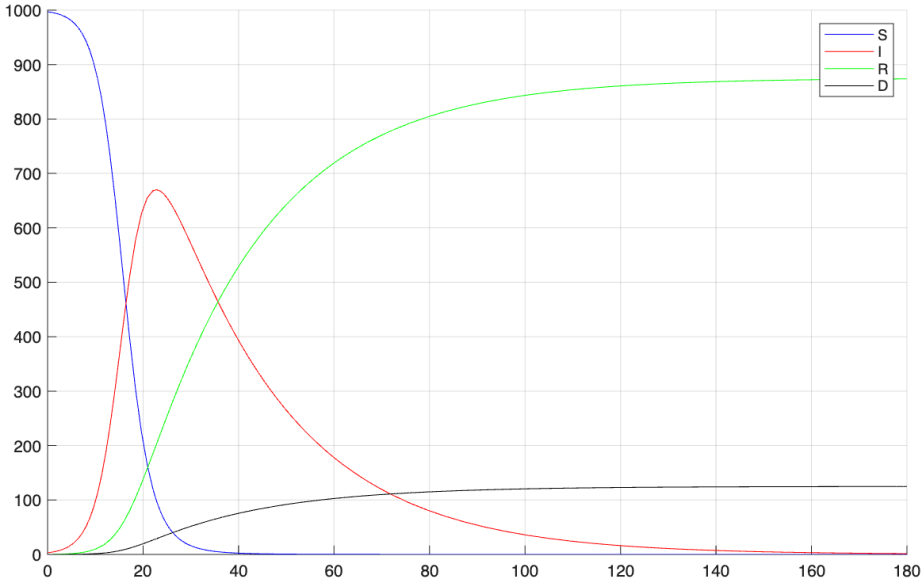

Figure 5: Numerical solutions of $S(t)$, $I(t)$, $R(t)$, and $D(t)$ against time

Analysis: On the Figure 5 the graph shows an epidemic curve. The number of *susceptible* people decreases pretty fast. The *infected* people line creates a peak in the beginning of the timeline. Eventually, the infection stops because there are no more *susceptible* people - there is no people for getting ill.

Most of the population is in the *recovered* group. A very small number of people is in the *deceased* group.

4.1.2 Task (b): MATLAB *ode45* (higher mortality rate)

Initial vector:

- $S(0) = 997$;
- $I(0) = 3$;
- $R(0) = 0$;
- $D(0) = 0$;
- Interval $= [0, 180]$ days.

Parameters:

- Total population $= 1000$;

- Infection rate $\beta = 0.4$;
- Recovery rate $\alpha = 0.035$;
- Mortality rate $\gamma = 0.05$.

Obtained results

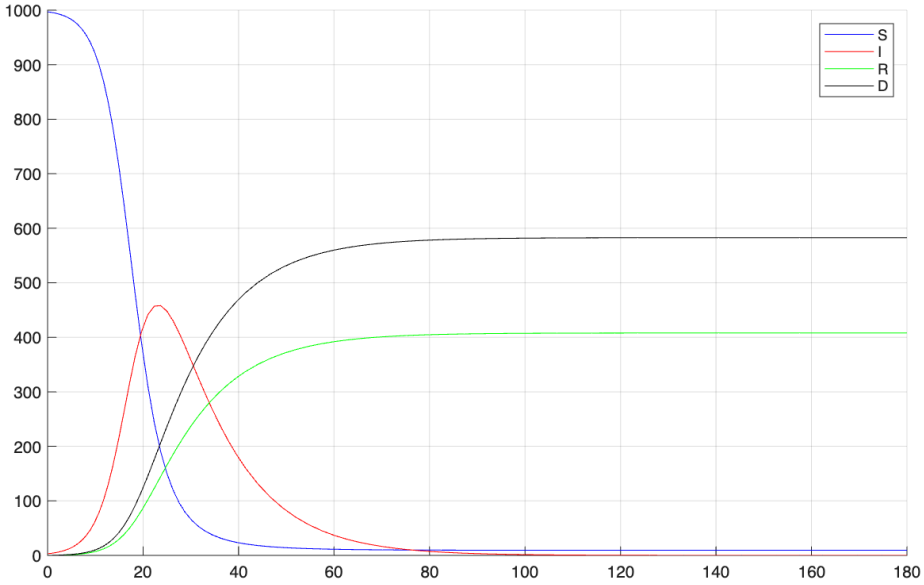

Figure 6: Numerical solutions of $S(t)$, $I(t)$, $R(t)$, and $D(t)$ against time

Analysis: On the Figure 6 the graph shows an epidemic curve. The number of *susceptible* people decreases pretty fast, but slightly slower than in the Part (a). The *infected* people line creates a peak in the beginning of the timeline. While the number *deceased* people increase significantly faster comparing to the previous part. Eventually, the infection stops because there are no more *susceptible* people.

Most of the population is in the *deceased* group in such model. Almost a half of the population is in the *recovered* group.

4.1.3 Task (c): Heun's method (small step size)

Initial vector:

- $S(0) = 997$;
- $I(0) = 3$;
- $R(0) = 0$;

- $D(0) = 0$;
- Interval = $[0, 180]$ days.

Parameters:

- Total population = 1000;
- Infection rate $\beta = 0.4$;
- Recovery rate $\alpha = 0.035$;
- Mortality rate $\gamma = 0.005$;
- Step size $\tau = 2$.

4.1.4 Theory of Heun's method

[1, page 297]

Heun's Method is a numerical technique used to solve differential equations. It is a small change to Euler's Method that makes the results much more accurate.

How it Works:

- Euler's Method finds the next point by using only the slope at the beginning of the step.
- Heun's Method improves this by using the average of two different slopes.
- First, the method uses Euler's formula to make a first prediction of the value at the right side of the interval.
- Next, it calculates the new slope at this predicted point.
- Finally, it takes the average of the starting slope and the predicted ending slope to get more accurate answer.

The Formula:
The calculation starts with an initial value, $w_0 = y_0$. To find the next approximation (w_{i+1}), it uses this equation:

$$w_{i+1} = w_i + \frac{h}{2}\Big(f(t_i, w_i) + f(t_i + h, w_i + hf(t_i, w_i))\Big) \tag{34}$$

Obtained results

Analysis: On the Figure 7 there is a graph which results after Heun's method calculations. Comparing it to the results in the Part (a) we can see that there is no actual difference. The lines on the graph from Part (a) are more smooth. However, overall the values are the same. We can say that our implemented Heun's method works as expected

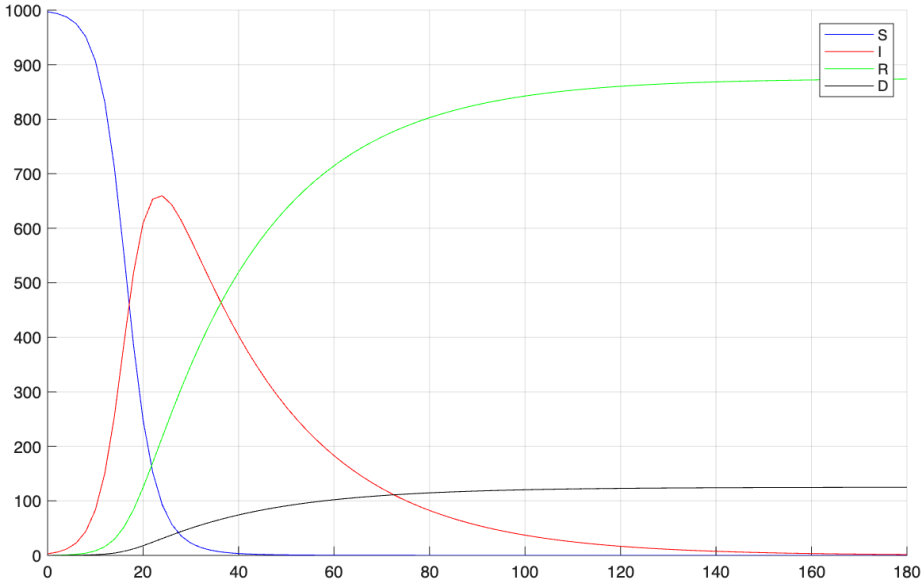

Figure 7: Numerical solutions of $S(t)$, $I(t)$, $R(t)$, and $D(t)$ against time

4.1.5 Task (d): Heun's method (big step size)

Initial vector:

- $S(0) = 997$;
- $I(0) = 3$;
- $R(0) = 0$;
- $D(0) = 0$;
- Interval $= [0, 180]$ days.

Parameters:

- Total population $= 1000$;
- Infection rate $\beta = 0.4$;
- Recovery rate $\alpha = 0.035$;
- Mortality rate $\gamma = 0.005$;
- Step size $\tau = 12$.

Obtained results

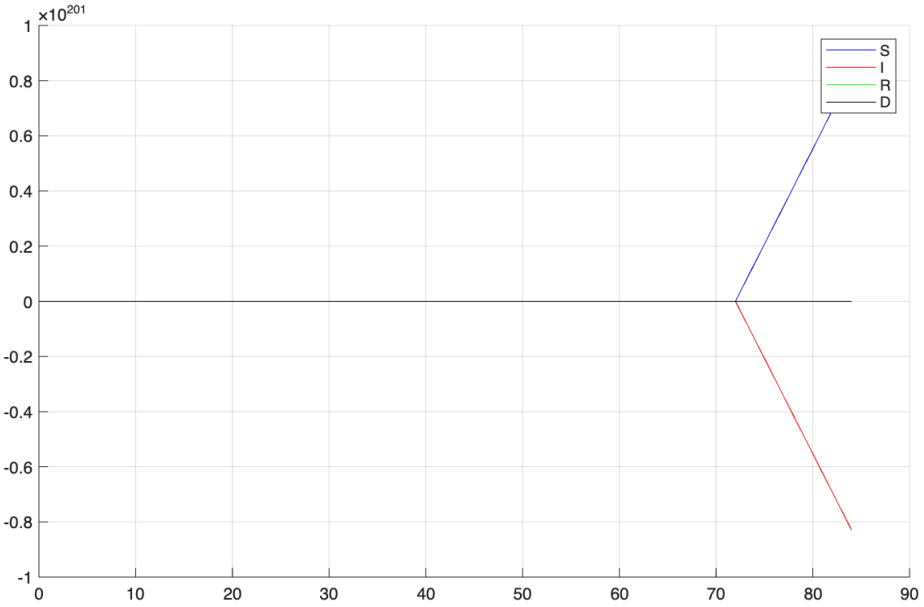

Figure 8: Numerical solutions of $S(t)$, $I(t)$, $R(t)$, and $D(t)$ against time

Analysis: On the Figure 8 we can observe total divergency. The only parameter that was changed comparing to the previous section - τ. It happens because

Explanation of unexpected results: We can explain this failure because the absence of stability. Some Numerical methods have bounded stability radius. This means that the calculation only remains accurate if the time step (τ) is small enough. If the step is not sufficiently small errors will multiply and grow extremely fast. Our chosen time step was outside this safe boundary. This causes the impossible results and the calculation breaking down.

4.2 Conclusions

- **Impact of parameters:** changing the parameters like mortality rate can significantly impact the results.
- **Heun's method:** this method demonstrated itself as a precise instrument to solve system of differential equations. However, step size is his weakness. It requires additional attention.
- **Instability:** with methods like Heun's the choose of the time step plays crucial role in the final results. If you choose the step out of bounds - errors will increase extremely fast and the solution becomes non-sense.

Declaration of AI Use

Uses of AI tools:

- AI was used for brainstorming and learning purposes on finding the order of convergence based on already existing materials from the textbook.
- AI was used for grammar editing in text

Students (Andrii Pozhylenkov and Dmytro Kuzko) retain full responsibility for all content.

AI was not used to generate solutions of problems, MATLAB codes, analysis and discussion of results.

References

References

[1] Timothy Sauer, Numerical analysis, Second edition, Pearson new international edition, 2014.

[2] J.H. Mathews & K.D. Fink, Numerical methods using MATLAB, 2003.

Appendix

You have to include all your MATLAB functions and scripts here. Do not include MATLAB codes as images from a print screen or similar. Enter your code using the syntax shown below.

MATLAB codes for Solving Nonlinear Algebraic Equation

```
% newtonraphson.m
function [p, n] = newtonraphson(f, df, p0, tol)
    errors = zeros(1000, 1);
    p_old = p0;

    for n = 1:1000
        if df(p_old) == 0
            break
        end

        p_new = p_old - f(p_old) / df(p_old);

        errors(n) = abs(p_new - p_old);

        if n > 2
            num = log(errors(n)) - log(errors(n-1));
            den = log(errors(n-1)) - log(errors(n-2));
            estimated_alpha = num / den;

            disp(['Step: ', num2str(n), ' Estimated Alpha = ',
                num2str(estimated_alpha)])
        end

        if errors(n) < tol
            p = p_new;
            return
        end

        p_old = p_new;

    end
end

% bisection.m
function [p, n] = bisection(f, a, b, tol)
    p = (a+b)/2;
    n = 1;

    while (b-a)/2 > tol
        f_a = f(a);
```

```
        f_p = f(p);

        if f_a * f_p < 0
            b = p;
        else
            a = p;
        end

        p = (a + b) / 2;
        n = n + 1;

    end
end

% secant.m
function [p, n] = secant(f, p0, p1, tol)
    errors = zeros(1000, 1);

    for n = 1:1000
        f_p1 = f(p1);
        f_p0 = f(p0);

        p = p1 - ((p1 - p0)*f_p1)/(f_p1 - f_p0);

        errors(n) = abs(p - p1);

        if n > 2
            num = log(errors(n)) - log(errors(n-1));
            den = log(errors(n-1)) - log(errors(n-2));
            estimated_alpha = num / den;

            disp(['Step: ', num2str(n), ' Estimated Alpha = ',
                num2str(estimated_alpha)])
        end

        if errors(n) < tol
            return
        end

        p0 = p1;
        p1 = p;
    end
end


% script.m
clear all; clc; format long;

f = @(x) -44.145*tan(x).^2 + 90*tan(x) - 43.145;
df = @(x) (90 - 88.29*tan(x))/cos(x).^2;
```

```
p0 = 0;
p1 = pi/4;
tol = 10^-15;

[bis_p, bis_n] = bisection(f, p0, p1, tol);

disp('Newton Raphson method with p0 = 0');
[new0_p, new0_n] = newtonraphson(f, df, p0, tol);
error_newtonraphson_p0_with_bisection = abs(bis_p - new0_p)

disp('Newton Raphson method with p0 = pi/4');
[new1_p, new1_n] = newtonraphson(f, df, p1, tol);
error_newtonraphson_p1_with_bisection = abs(bis_p - new1_p)

disp('Secant method with p0, p1 = 0, pi/4');
[sec_p, sec_n] = secant(f, p0, p1, tol);
error_secant_with_bisection = abs(bis_p - sec_p)

disp(['Bisection method: p = ', num2str(bis_p, 16), ', iterations = ',
    num2str(bis_n)]);
disp(['Newton-Raphson method (0): p = ', num2str(new0_p, 16), ',
    iterations = ', num2str(new0_n)]);
disp(['Newton-Raphson method (pi/4): p = ', num2str(new1_p, 16), ',
    iterations = ', num2str(new1_n)]);
disp(['Secant method: p = ', num2str(sec_p, 16), ', iterations = ',
    num2str(sec_n)]);
```

MATLAB codes for Numerical Differentiation & Numerical Integration

```
% cdf2.m
function [f] = cdf2(y, h)
    len = length(y);

    f = zeros(1, len);
    f(1) = (2*y(1) - 5*y(2) + 4*y(3) - y(4))/h^2;
    f(end) = (2*y(end) - 5*y(end-1) + 4*y(end-2) - y(end-3))/h^2;

    for i = 2:len-1
        f(i) = (y(i-1) - 2*y(i) + y(i+1))/h^2;
    end
end

% trapezoidalsum.m
function T = trapezoidalsum(f, N, h)
```

```
    T = f(1) + f(end);

    for k = 2:N
        T = T + 2*f(k);
    end

    T = T * (h/2);
end

% simpsonsum.m
function S = simpsonsum(f, N, h)
    S = f(1) + f(end);

    for k = 2:N
        if mod(k - 1, 2) == 0
            S = S + 2*f(k);
        else
            S = S + 4*f(k);
        end
    end

    S = S * h / 3;
end

% script.m
clear all; clc; format long;

x = 1:0.1:2;
y = [0.4, 0.752, 1.236, 1.864, 2.648, 3.6, 4.732, 6.056, 7.584, 9.328,
     11.3];

a = x(1);
b = x(end);
N = length(x) - 1;
h = (b - a) / N;

f2 = cdf2(y, h)
lnf2 = log(f2)

trapezoidalsum = trapezoidalsum(lnf2, N, h)
simpsonsum = simpsonsum(lnf2, N, h)

exact = 2.871201010907891
trap_err = abs(exact - trapezoidalsum)
simp_err = abs(exact - simpsonsum)
```

MATLAB codes for Interpolation & Curve Fitting

```
% newton_matrix.m
function p = newton_matrix(x, y)
    x = x(:);
    y = y(:);
    n = length(x);

    A = zeros(n);
    A(:, 1) = y;

    for j = 2:n
        for i = j:n
            A(i, j) = (A(i, j-1) - A(i-1, j-1)) / (x(i) - x(i-j+1));
        end
    end
    display(A);

    p = zeros(n, 1);
    for i = 1:n
        p(i) = A(i, i);
    end
    display(p);
end

% newton_polynomial.m
function f = newton_polynomial(p, t, x)
    n = length(p) - 1;
    f = p(1);
    product = x - t(1);

    for i = 1:n
        f = f + p(i+1) .* product;
        product = product .* (x - t(i+1));
    end
end

% interpolation.m
clear all; clc; format long;

t = [4 8 12 16 20 24];
c = [1600 1320 1000 890 650 560];

p = newton_matrix(t,c);

x = linspace(0, 24, 1000);
f = newton_polynomial(p, t, x);
f0 = f(1);

figure; grid on; hold on;
plot(x, f, 'b');
```

```
xlabel('Time');
ylabel('Value');

plot(0, f0, 'rx', 'MarkerSize', 10);
plot(t, c, 'ro');
hold off;

% solve_normal.m
function p = solve_normal(x, y, n)
    x = x(:);
    y = y(:);

    powers = n:-1:0;
    A = x .^ powers;

    p = (A' * A) \ (A' * y);
end

% curvefitting.m
clc; clear all; format long;

t = [4 8 12 16 20 24];
c = [1600 1320 1000 890 650 560];

t0 = 0;
linpoints = linspace(0, 24, 1000);

%linear
coeffs_l = solve_normal(t, c, 1);
display(coeffs_l);

predicted_l = polyval(coeffs_l, t);
predicted_l_value = polyval(coeffs_l, t0);
predicted_l_plot = polyval(coeffs_l, linpoints);

%exponential
coeffs_e = solve_normal(t, log(c), 1);
display(coeffs_e);

a_exp = exp(coeffs_e(2));
display(a_exp);

b_exp = coeffs_e(1);
display(b_exp);

predicted_e = a_exp * exp(b_exp * t);
predicted_e_value = a_exp * exp(b_exp * t0);
predicted_e_plot = a_exp * exp(b_exp * linpoints);

rms_l = rms(predicted_l - c);
```

```
rms_e = rms(predicted_e - c);

disp(['t=0 approx (linear): ', num2str(predicted_l_value)])
disp(['t=0 approx (exp): ', num2str(predicted_e_value)])
disp(['rms error (linear): ', num2str(rms_l)])
disp(['rms error (exp): ', num2str(rms_e)])

figure; hold on; grid on;
plot(t, c, 'ro');
plot(linpoints, predicted_l_plot, 'b-');
plot(linpoints, predicted_e_plot, 'g-');

xlabel('Time');
ylabel('Concentration');
legend('Data', 'Linear', 'Exponential');
hold off;
```

MATLAB codes for Numerical Solution of Differential Equations

```
% A_and_B.m
clear all; clc; format long;

tspan = [0 180];
pop = 1000;
beta = 0.4;
alpha = 0.035;
gamma = 0.005;
% gamma = 0.05;

F = @(t, Y) [
    -(beta/pop) * Y(2) * Y(1);
    (beta/pop) * Y(2) * Y(1) - alpha * Y(2) - gamma * Y(2);
    alpha * Y(2);
    gamma * Y(2)
];

Y0 = [997; 3; 0; 0];

[t, y] = ode45(F, tspan, Y0);

S = y(:,1);
I = y(:,2);
R = y(:,3);
D = y(:,4);
```

```
figure; hold on; grid on;
plot(t, S, 'b')
plot(t, I, 'r')
plot(t, R, 'g')
plot(t, D, 'w')

legend('S', 'I', 'R', 'D')
hold off;

% heun.m
function [t, y] = heun(f, a, b, y0, N)
    tau = (b - a) / N;

    m = length(y0);
    y = zeros(N+1, m);
    y(1, :) = y0;

    t = linspace(a, b, N+1)';

    for n = 2:(N+1)
        y_prev = y(n-1, :)';

        k1 = f(t(n-1), y_prev);
        y_temp = y_prev + tau * k1;
        k2 = f(t(n), y_temp);

        y(n, :) = (y_prev + (tau / 2) * (k1 + k2))';
    end
end

% C_and_D.m
clear all; clc; format long;

a = 0;
b = 180;
tau = 2;
% tau = 12;
N = (b - a) / tau;

beta = 0.4;
alpha = 0.035;
gamma = 0.005;
pop = 1000;

F = @(t, y) [
    -(beta/pop) * y(2) * y(1);
    (beta/pop) * y(2) * y(1) - alpha * y(2) - gamma * y(2);
    alpha * y(2);
    gamma * y(2)
];
```

```
y0 = [997; 3; 0; 0];

[t, y] = heun(F, a, b, y0, N);

S = y(:,1);
I = y(:,2);
R = y(:,3);
D = y(:,4);

figure; hold on; grid on;
plot(t, S, 'b')
plot(t, I, 'r')
plot(t, R, 'g')
plot(t, D, 'w')

legend('S', 'I', 'R', 'D')
hold off;
```